Defensa del TFM
Guion y hoja de ruta

Juan Muñoz Rovira · Seguimiento de personas y navegación autónoma

01	Seguimiento de personas	0:45	**0:45**
02	Punto de partida	1:10	**1:55**
03	Objetivos y estado	0:50	**2:45**
04	Plataforma económica	0:55	**3:40**
05	Arquitectura	1:20	**5:00**
06	Interacción por gestos	1:00	**6:00**
07	▶ Arranque y parada	0:30	**6:30**
08	Detección y fusión	1:15	**7:45**
09	Seguimiento y seguridad · + ficha «por dentro»	1:15	**9:00**
10	▶ Obstáculos sólidos	0:40	**9:40**
11	6 / 7 metas alcanzadas · + ficha «por dentro»	1:10	**10:50**
12	▶ Navegación autónoma	0:40	**11:30**
13	El gesto «casa»	1:15	**12:45**
14	▶ Gesto «casa»	1:05	**13:50**
15	9 / 11 a casa	1:10	**15:00**
16	Medir, no suponer	0:50	**15:50**
17	Límites y próximos pasos	1:00	**16:50**
18	Cuatro conclusiones	0:50	**17:40**
19	Muchas gracias	0:10	**17:50**

- Objetivo: **≈ 18 min** de un turno de ~20 · cronómetro a la vista
- Cada ficha: **qué decir** (guion completo) + **para entenderlo** (lo que hay detrás). Tacha lo que no vayas a decir
- Fichas «por dentro» de la 9 y la 11: para estudiar y para las preguntas, no para leer en la exposición
- Vídeos 2 y 3 recortados a ~30–40 s · reserva 20–27: número + Intro

POR DENTRO

11 SLAM, AMCL y Nav2, por dentro

PARA ESTUDIAR Y PARA LAS PREGUNTAS · 1 DE 2

Tres problemas distintos: hacer el mapa, saber dónde estoy y llegar a la meta.

1. **A · El mapa (una vez) — SLAM Toolbox:** moví el robot con el teclado por el laboratorio. Con cada escaneo y la odometría, SLAM Toolbox llena una rejilla de ocupación de 5 cm por celda (libre, ocupado, desconocido). Al reconocer una zona ya visitada «cierra el bucle» y corrige la deriva acumulada. El mapa se guarda y se reutiliza.
2. **B · Dónde estoy (siempre) — AMCL:** filtro de partículas: entre 500 y 2000 hipótesis de pose (x, y, θ) sobre el mapa. Al moverse, cada partícula se desplaza según la odometría, con ruido; con cada escaneo se compara lo que «vería» cada una con lo medido y se le da un peso; al remuestrear, se concentran en la pose real. Resultado: la transformación map→odom, que corrige la deriva. Se actualiza cada 0,25 m o 0,2 rad.
3. **C1 · La meta:** llega una acción NavigateToPose (desde RViz o desde control_node con el gesto «casa»). bt_navigator sigue un árbol de comportamiento: planificar → seguir → si falla, recuperar.

01 Seguimiento de personas

≈ 0:45 · ACABA EN 0:45

QUÉ DECIR

Buenos días, miembros del tribunal. Me llamo Juan Muñoz Rovira y voy a presentar mi Trabajo Fin de Máster, dirigido por Enric Cervera: la segunda parte de un sistema de seguimiento de personas sobre un robot móvil.

En resumen: partí de un TurtleBot 2 que solo sabía seguir a una persona y lo he convertido en un pequeño asistente móvil. Entiende gestos, construye y usa un mapa del laboratorio, navega solo y combina los dos comportamientos. El mapa de la derecha es el resultado final; volveré a él al final.

> **Idea clave:** *Preséntate y da la idea en una frase: de «solo sigue a una persona» a «entiende gestos, usa un mapa y navega solo».*

PARA ENTENDERLO

- **Qué es el TFM:** la Parte 2 de un sistema de seguimiento de personas en un TurtleBot 2. Añade gestos, un mapa SLAM, navegación autónoma con Nav2 y la integración de todo con el gesto «casa».
- **El mapa de la portada** es la Fig. 5.5: las vueltas a casa reales del 23/09 sobre el mapa del laboratorio (verde = éxito, rojo = fallo, estrella = casa).
- **Tutor:** Enric Cervera Mateu · Máster Universitario en Sistemas Inteligentes · Universitat Jaume I.
- Primera frase despacio y mirando al tribunal; es la que marca el ritmo.

POR DENTRO

11 SLAM, AMCL y Nav2, por dentro

PARA ESTUDIAR Y PARA LAS PREGUNTAS · 2 DE 2

4. **C2 · Ruta global — NavFn:** planner_server busca el camino de menor coste (tipo Dijkstra) en el costmap global: el mapa + los obstáculos + un inflado de 0,55 m alrededor de cada uno, para que el robot (radio 0,18 m) pase holgado.
5. **C3 · Seguimiento local — DWB:** controller_server, a 10 Hz, prueba 20 × 20 combinaciones de velocidad lineal y angular, simula cada una 1,7 s y la puntúa con «críticos» (seguir la ruta, acercarse a la meta, alejarse de obstáculos, no oscilar); se queda con la mejor. Usa un costmap local de 3 × 3 m que se actualiza con el LiDAR: por eso esquiva obstáculos que no están en el mapa.
6. **C4 · Llegada y atascos:** meta alcanzada a menos de 0,25 m y 0,25 rad (≈ 14°). Si se atasca, behavior_server prueba recuperaciones: girar, retroceder, esperar.
7. **D · Salida:** la velocidad de DWB va a /nav2/cmd_vel; control_node solo la pasa a la base en HOMING.

> **La anécdota de DDS.** En ROS 2 los nodos se encuentran entre sí mediante DDS, y ese descubrimiento tarda un poco. Mis comandos de prueba eran tan cortos que terminaban antes: AMCL no recibía nada y parecía «congelado». Con comandos más largos convergió sin problemas.

02 Punto de partida

≈ 1:10 · ACABA EN 1:55

QUÉ DECIR

Este trabajo parte de un TFM anterior, de Joaquín Martínez Miralles. Su robot detectaba a la persona con un LiDAR 2D: agrupaba los puntos del escaneo con DBSCAN, buscaba el patrón de dos piernas y las seguía. La figura muestra lo que produce ese tipo de detección: cada punto es una posición de la persona, medida por el LiDAR en el sistema del robot, en una de mis tomas.

Funcionaba, pero tenía cuatro carencias claras. Sin mapa, no había navegación planificada. Sin localización global, al perder a la persona no tenía ninguna referencia para reorientarse. El usuario no tenía una forma clara de arrancarlo o pararlo. Y la cámara estaba montada, pero no participaba en el lazo de control.

Mi TFM añade gestos, un mapa y navegación autónoma, y hace que funcionen juntos.

Idea clave: *Parte 1 = seguir piernas con el LiDAR. Cuatro carencias: sin mapa, sin localización, sin control claro, cámara sin uso. Tu TFM las cubre.*

PARA ENTENDERLO

- **DBSCAN** agrupa los puntos del escaneo por densidad: une los que están a menos de *eps* entre sí y no necesita saber cuántos grupos hay. Cada pierna sale como un grupo pequeño.
- **Patrón de dos piernas:** dos grupos del tamaño de una pierna, a una separación plausible → su punto medio es la posición de la persona.
- **La figura** (Fig. 3.2) son posiciones de la persona en el sistema del robot, coloreadas por tiempo, de una de mis tomas: el dato con el que trabaja todo el sistema.
- **Cada carencia se convierte en un objetivo:** sin control claro → gestos · sin mapa → SLAM · sin localización → AMCL y Nav2 · cámara sin uso → fusión LiDAR–cámara.

4 carencias

VÍDEO 3 · 30–40 S

12 Navegación autónoma

≈ 0:40 · ACABA EN 11:30

QUÉ DECIR

Aquí las metas se fijan sobre el mapa y el robot se desplaza solo.

▶ **Reproducir el vídeo.** Cuando encuentra un obstáculo que no está en el mapa, lo ve con el LiDAR y lo rodea.

Idea clave: *Si sale RViz en el monitor, señálalo: «este es el mapa y la ruta planificada».*

PARA ENTENDERLO

- Vídeo recortado a ~30–40 s.
- En el monitor (RViz): el mapa en gris, la ruta planificada y los puntos del LiDAR. El obstáculo no mapeado aparece en el costmap local y la trayectoria se desvía.
- Nav2 va a 0,26 m/s como máximo y da la meta por alcanzada a menos de 0,25 m y ≈ 14°.

03 Objetivos y estado

≈ 0:50 · ACABA EN 2:45

QUÉ DECIR

El objetivo general era un asistente móvil que navegue solo y entienda al usuario. Se desglosa en cinco objetivos específicos y uno exploratorio.

Cuatro están cumplidos y validados en el robot real: la interacción por gestos, el mapa SLAM del laboratorio, la navegación autónoma con Nav2 —**seis de siete** objetivos de navegación— y la fusión LiDAR–cámara.

El quinto, el guiado del usuario, tiene un primer paso validado: la vuelta a casa con un gesto, con **nueve de once** vueltas completadas. El guiado a varios destinos y el objetivo exploratorio de reconocer salas con códigos QR quedan como trabajo futuro.

> **Idea clave:** *4 de 5 objetivos cumplidos; el 5.º (guiado) tiene un primer paso validado (gesto «casa», 9/11); QR no abordado.*

PARA ENTENDERLO

- **Los 5 específicos:** (1) interacción por gestos · (2) mapa con SLAM · (3) navegación autónoma con Nav2 · (4) fusión LiDAR–cámara · (5) guiado del usuario. **Exploratorio (6):** reconocer salas con códigos QR.
- **«Completado»** = funcionando y validado en el robot real, con datos grabados.
- **El 5.º** pedía acompañar al usuario a destinos del edificio; la vuelta a «casa» es un destino fijo: el primer paso, validado con 9 de 11.
- **6 de 7** = prueba de Nav2 del 27/07 · **9 de 11** = vueltas a casa del 23/09.
- Si preguntan por el QR: sesiones de laboratorio limitadas; se priorizó integrar seguimiento y navegación.

4 de 5 | **6 / 7 metas** | **9 / 11 a casa**

13 El gesto «casa»

≈ 1:15 · ACABA EN 12:45

QUÉ DECIR

Aquí es donde todo se junta: el gesto «casa». El diagrama es la máquina de estados de control_node.

Con la mano derecha se pasa de IDLE a TRACKING y el robot me sigue. Con el tejado, de TRACKING a HOMING: control_node envía a Nav2 una meta NavigateToPose hacia la posición de casa y, a partir de ahí, solo la velocidad de Nav2 llega a la base. Al llegar, si Nav2 falla o con la mano izquierda, vuelve a IDLE.

Un detalle fue muy importante: el seguidor inactivo sigue publicando velocidad cero diez veces por segundo. Reenviar eso habría anulado a Nav2; por eso el arbitraje se hace por estado, y no con la regla de «gana el último mensaje».

> **Idea clave:** *Recorre la máquina de estados (IDLE → TRACKING → HOMING → IDLE). Detalle clave: el seguidor publica velocidad cero a 10 Hz; por eso se arbitra por estado.*

PARA ENTENDERLO

- **Secuencia HOMING:** gesto tejado → control_node desactiva el seguidor (servicio enable_tracking) → envía la acción NavigateToPose con la pose «casa» guardada en el mapa → reenvía solo /nav2/cmd_vel.
- **Vuelta a IDLE:** Nav2 informa de llegada (éxito) · Nav2 aborta (sin ruta) · mano izquierda → control_node cancela la meta.
- **Por qué por estado:** un multiplexor de «gana el último mensaje» dejaría pasar los ceros que el seguidor sigue publicando a 10 Hz, y Nav2 no conseguiría mover el robot.
- **El diagrama** es una versión simplificada de la Fig. 3.3; la completa está en la reserva (diapositiva 21).

IDLE → TRACKING → HOMING

04 Plataforma económica

≈ 0:55 · ACABA EN 3:40

QUÉ DECIR

La plataforma es deliberadamente de bajo coste. La base Kobuki es de tracción diferencial; sus encoders de rueda y su giróscopo dan la odometría.

El RPLIDAR A2M8 barre **360 grados** a unos **10 hercios**. Está montado a unos **47 centímetros** del suelo, así que su plano de escaneo corta a las personas a la altura de las piernas. Eso es lo que aprovecha el detector y, como veremos, también su principal limitación.

Sustituí la webcam original por una SPCA2650 para encuadrar los gestos a un metro, que es la distancia real de seguimiento. Todo se ejecuta a bordo, en un único Intel NUC con Ubuntu 24.04 y ROS 2 Jazzy, con una carga de alrededor del **30 %** por núcleo.

> **Idea clave:** *Kobuki + LiDAR a 47 cm (altura de piernas: ventaja y limitación) + cámara SPCA2650 + NUC al ~30 % por núcleo.*

PARA ENTENDERLO

- **Kobuki:** dos ruedas motrices (tracción diferencial): avanza y gira, pero no se desplaza de lado. La odometría (encoders + giróscopo) deriva con el tiempo → por eso hace falta AMCL para situarse en el mapa.
- **RPLIDAR A2M8:** gira 360° unas 10 veces por segundo; ~300–400 puntos útiles por vuelta; alcance ~12 m. Mide un solo plano a 0,47 m: ve piernas, pero no patas finas de silla ni lo que quede por encima.
- **Cámara:** la Logitech C270 original no encuadraba a la persona a 1 m (el detector HOG necesitaba el cuerpo entero). La SPCA2650, más angular, ve brazos y cabeza a esa distancia.
- **Intel NUC sin GPU:** todo corre en CPU. De ahí las decisiones: MediaPipe (ligero), DBSCAN propio, Kalman lineal.
- Seguimiento: distancia objetivo **1,0 m**, velocidad máxima **0,18 m/s**. Nav2: hasta **0,26 m/s**.

~10 Hz	~47 cm	~1 m	~30 % por núcleo

VÍDEO 4 · 1:02

14 Gesto «casa»

≈ 1:05 · ACABA EN 13:50

QUÉ DECIR

Esta es la secuencia completa en una sola toma.

▶ **Reproducir el vídeo.** Mano derecha: me sigue. Mano izquierda: se para. El tejado: pasa el control a Nav2 y vuelve a casa solo. A mitad de camino levanto la mano izquierda y la vuelta se cancela al instante. Otra vez el tejado, y termina el trayecto a casa.

> **Idea clave:** *Narra la secuencia mientras ocurre: derecha, izquierda, tejado, cancelar a mitad, tejado otra vez.*

PARA ENTENDERLO

- Vídeo de 1:02.
- Orden: derecha (sigue) → izquierda (para) → tejado (vuelve) → izquierda a mitad (cancela al instante) → tejado otra vez (llega a casa).
- La cancelación a mitad de camino es la que se midió: Nav2 la confirma en ~20 ms.

05 Arquitectura

≈ 1:20 · ACABA EN 5:00

QUÉ DECIR

El software es un conjunto de nodos ROS 2 independientes que solo se comunican mediante topics, servicios y acciones.

Siguiendo los datos: el LiDAR publica sus escaneos; detection_node encuentra las piernas con DBSCAN y un filtro geométrico y publica la posición de la persona; tracking_node la filtra con un filtro de Kalman, calcula una velocidad y evita obstáculos. El nodo de la cámara, visual_detection_node, aporta dos cosas: los gestos y una dirección aproximada de la persona.

La pieza clave es control_node. Es una máquina de estados y es el único nodo cuya salida llega a las ruedas: en cada momento reenvía la velocidad del seguidor, la de Nav2 o la del teclado, nunca dos a la vez. Este arbitraje es lo que después permite que seguimiento y navegación convivan.

> **Idea clave:** *Sigue el flujo de datos en la figura. Mensaje clave: control_node es el único que manda en las ruedas (árbitro).*

PARA ENTENDERLO

- **ROS 2:** cada nodo es un proceso independiente. **Topics** = flujo continuo publicar/suscribir (/scan, /person_position) · **servicios** = petición/respuesta (enable_tracking) · **acciones** = tareas largas con realimentación (NavigateToPose).
- **Cadena:** /scan → detection_node → /person_position → tracking_node → /tracking/velocity_cmd → control_node → /cmd_vel → base Kobuki.
- **Cámara:** visual_detection_node publica los gestos (/gesture_command) y el rumbo de la persona, que detection_node usa en el respaldo.
- **Nav2** publica en /nav2/cmd_vel (remapeado), nunca directamente a la base.
- **Estados de control_node:** INIT, IDLE, TRACKING, HOMING, MANUAL (teclado) y SHUTDOWN (apagado coordinado).
- collision_handling_node y user_interface_node también existen (seguridad extra y telemetría); conectar el primero con control_node es trabajo futuro.

control_node = árbitro

15 9 / 11 a casa

≈ 1:10 · ACABA EN 15:00

QUÉ DECIR

En la sesión final de laboratorio, el 23 de septiembre, pedí **doce vueltas** a casa: una la cancelé a propósito y once fueron intentos reales.

Se completaron **nueve** —un **82 %**— en **15,5 segundos** de media, con un error final medio de **27 centímetros** y **8 grados**, muy cerca de la propia tolerancia de meta de Nav2. La cancelación con la mano izquierda se confirmó en **20 milisegundos**. El mapa muestra las trayectorias grabadas; la estrella es casa.

Los dos fallos no los causó el gesto ni la máquina de estados: AMCL, con una incertidumbre de medio metro, situó al robot dentro de la zona de seguridad de un obstáculo, el planificador no encontró una ruta válida y el robot volvió de forma segura a IDLE.

> **Idea clave:** *9/11 (82 %), 15,5 s, 0,27 m / 8°, cancelación en 20 ms. Los 2 fallos: localización junto a un obstáculo, no el gesto; el robot queda seguro en IDLE.*

PARA ENTENDERLO

- **23/09:** 12 peticiones = 1 cancelada a propósito + 11 intentos; 9 completadas (82 %).
- **Tiempo** gesto → en casa: 15,5 s de media (11–21 s). **Error de llegada:** 0,27 m / 8°, junto a la tolerancia de meta de Nav2 (0,25 m / 0,25 rad ≈ 14°).
- **Cancelación:** Nav2 confirma en ~20 ms; el robot avanza ~7 cm más.
- **Por qué fallaron 2:** AMCL (σ ≈ 0,5 m) colocó al robot dentro de la zona inflada que rodea a un obstáculo (inflado de 0,55 m). Desde ahí NavFn no encuentra ruta → Nav2 aborta → control_node vuelve a IDLE, de forma segura.
- **Solución propuesta:** una maniobra de salida antes de pedir la ruta y un mejor ajuste de AMCL.

9 de 11 (82 %) | **15,5 s de media** | **20 ms**

06 Interacción por gestos

≈ 1:00 · ACABA EN 6:00

QUÉ DECIR

El usuario se comunica con el robot mediante tres gestos, reconocidos a partir del esqueleto de MediaPipe Pose. Mano derecha arriba: el robot empieza a seguir. Mano izquierda arriba: se detiene, o cancela una vuelta a casa. Las dos manos juntas sobre la cabeza, como un tejado: el robot vuelve a casa.

Cada gesto debe mantenerse durante varios fotogramas, con una pausa entre órdenes, para que los movimientos casuales no disparen nada: en unos cuatro minutos de gesticulación libre no hubo ningún falso positivo.

Elegí el tejado porque es simétrico, así que funciona también de espaldas; y mientras el robot sigue, la persona está de espaldas a él. Se reconoció de espaldas en los dos intentos.

> **Idea clave:** *Tres gestos (derecha arranca, izquierda para o cancela, tejado = casa). Confirmación en varios fotogramas → 0 falsos positivos. Tejado simétrico → funciona de espaldas.*

PARA ENTENDERLO

- **MediaPipe Pose** (BlazePose) da 33 puntos clave del cuerpo por imagen, en tiempo real y en CPU; se usan hombros, muñecas, caderas y nariz.
- **Reglas:** inicio = muñeca derecha por encima del hombro derecho · parada = lo mismo con la izquierda · «casa» = las dos muñecas juntas por encima de la cabeza.
- **Robustez:** el margen es proporcional a la altura del torso (no píxeles fijos); cada gesto se confirma durante varios fotogramas y hay un tiempo de espera entre órdenes.
- **Por qué el tejado:** simétrico → se ve igual de frente y de espaldas; no aparece al caminar; los brazos cruzados se descartaron porque las muñecas se tapan.
- **Datos:** 0 falsos positivos en ~4 min de gesticulación libre; tejado de espaldas 2 de 2 (algo más lento: la cabeza tapa una muñeca).

3 gestos | **0 falsos positivos en ~4 min** | **de espaldas 2/2**

16 Medir, no suponer

≈ 0:50 · ACABA EN 15:50

QUÉ DECIR

Una palabra sobre el método, porque marcó todo el proyecto. Cada toma se grabó como rosbag en el NUC, se convirtió a CSV y se analizó fuera de línea: métricas como el MAE, el RMS y el porcentaje de detección, y cuatro figuras estándar, como esta de distancia.

Cada cambio se comprobó con datos grabados, no a ojo. Y hacía falta: los errores de la izquierda —un signo invertido en el controlador PD, un sector de evitación invertido, un factor de reducción que nunca llegaba a cero y la latencia de DDS— solo aparecieron en el robot real.

> **Idea clave:** *Método: rosbag → CSV → métricas y 4 figuras; cada cambio comprobado con datos. Los errores solo salieron en el robot real.*

PARA ENTENDERLO

- **rosbag:** grabación de los topics de ROS 2 durante cada toma, en el propio NUC.
- **Métricas:** MAE y RMS de la distancia (error frente a la distancia objetivo), % del tiempo con la persona detectada, nº de pérdidas, % de saltos > 0,8 m y % de saturación de la velocidad angular.
- **4 figuras estándar** por toma: distancia, ángulo, velocidades y trayectoria (odometría).
- **Errores que solo salieron en el robot:** signo del PD invertido (giraba al revés), sector de evitación invertido, factor de reducción que no llegaba a 0, latencia de DDS.
- Las últimas correcciones también se comprobaron fuera de línea con el código real de los nodos y escenarios sintéticos.

rosbag → CSV → métricas

VÍDEO 1 · 28 S

07 Arranque y parada

≈ 0:30 · ACABA EN 6:30

QUÉ DECIR

Aquí lo vemos en directo, en el laboratorio.

▶ **Reproducir el vídeo.** Mano derecha: el robot empieza a seguirme… y mano izquierda: se detiene.

Idea clave: *Deja hablar al vídeo; nombra solo los dos gestos cuando aparezcan.*

PARA ENTENDERLO

- Vídeo de 28 s, sesión real en el laboratorio.
- Al levantar la mano derecha el robot tarda un instante en arrancar: es la confirmación de varios fotogramas, hecha a propósito para evitar falsos positivos.
- Si el tribunal pregunta por ese retardo, esa es la respuesta.

17 Límites y próximos pasos

≈ 1:00 · ACABA EN 16:50

QUÉ DECIR

Los resultados también tienen límites claros. En percepción: un único plano LiDAR a **47 centímetros** no vio las patas finas de una silla, y la evitación reactiva no distingue a la persona de un obstáculo. En validación: muestras pequeñas, una sola sesión para Nav2 y otra para el gesto «casa», en un único laboratorio; y la localización junto a obstáculos causó los dos fallos de vuelta a casa.

Los próximos pasos: guiado a varios destinos, con un gesto o un código QR para cada uno; una maniobra de salida antes de replanificar; una cámara RGB-D o un segundo LiDAR; conectar collision_handling_node con control_node; y pruebas con más personas y entornos.

Idea clave: *Sé honesto: LiDAR de un plano, muestras pequeñas, una sesión, un laboratorio. Próximos pasos: varios destinos (gesto/QR), maniobra de salida, RGB-D.*

PARA ENTENDERLO

- **Percepción:** un único plano LiDAR a 0,47 m → no ve patas finas ni asientos; la evitación reactiva trata a la persona como obstáculo si se acerca mucho.
- **Validación:** Nav2 y el gesto «casa», una sesión cada uno; un solo laboratorio; varias métricas de una sola toma → primeros resultados, no estadística.
- **Próximos pasos:** guiado a varios destinos (un gesto o QR por destino) · maniobra de salida antes de replanificar · cámara RGB-D Orbbec Astra (ya disponible) o 2.º LiDAR · conectar collision_handling_node · más personas y entornos.
- Reconocer los límites suma: todos están en la memoria (§5.9 y §5.10).

08 Detección y fusión

≈ 1:15 · ACABA EN 7:45

QUÉ DECIR

La detección funciona en dos etapas. Primero, el LiDAR: cada escaneo tiene entre 300 y 400 puntos. Se filtran, se interpolan y se agrupan con DBSCAN; lo reimplementé sobre un KD-tree porque scikit-learn no era compatible con el Python del NUC. Después, los clústeres se filtran por tamaño, radio y forma para encontrar un par de piernas.

Segundo, el respaldo: cuando no hay un par válido —por ejemplo, cuando la persona se pone de perfil— el rumbo de la cámara elige el clúster del LiDAR mejor alineado con la persona. Es una fusión asimétrica: el LiDAR da la posición y la cámara solo deshace el empate, así que no hace falta una calibración precisa.

Con el robot parado, la detección fue continua, del **100 %**, con unos seis grados entre el rumbo de la cámara y el clúster elegido. Antes de este respaldo, el seguimiento se detenía tras **30 segundos** sin piernas.

> **Idea clave:** *LiDAR primero (DBSCAN propio + filtro de piernas); si no hay par, la cámara elige el clúster. Fusión asimétrica, sin calibración precisa. 100 % en estático, ~6°.*

PARA ENTENDERLO

- **Cadena de detección:** /scan → filtro de mediana + interpolación → DBSCAN (eps 0,12 m, mín. 4 puntos) → filtro geométrico (tamaño, radio, forma) → emparejar dos piernas → filtro temporal → /person_position.
- **eps** = distancia máxima entre vecinos para estar en el mismo grupo; se calibró con la resolución angular real del LiDAR.
- **Filtro temporal:** persistencia (varios barridos para confirmar o dar por perdida), filtro de deriva (descarta saltos lejos de la posición confirmada) y confirmación estable.
- **Respaldo con cámara:** rumbo = punto medio de los hombros en la imagen → se pasa al sistema del láser → clúster con menor desviación angular → se confirma durante varios barridos.
- **Fusión asimétrica:** el LiDAR mide la distancia con precisión; la cámara solo aporta la dirección. Basta el campo de visión horizontal y un signo: sin calibración extrínseca.
- **25/06 (robot parado):** 100 % de detección, 0 pérdidas, ≈ 6° entre el rumbo de la cámara y el clúster elegido.

eps 0,12 m | **100 % en estático** | **~6° cámara–clúster**

18 Cuatro conclusiones

≈ 0:50 · ACABA EN 17:40

QUÉ DECIR

Para terminar, cuatro conclusiones. Objetivos: **cuatro de cinco** cumplidos, más un primer paso que funciona hacia el guiado del usuario. Arbitraje: control_node hace que seguimiento y Nav2 convivan, y un gesto alterna entre ellos. Bajo cómputo: todo corre a bordo, a un **30 %** por núcleo, gracias a decisiones como un DBSCAN propio, una fusión sin calibración precisa y un filtro de Kalman lineal. Y la lección principal: probar en el robot real y medir cada cambio.

La aportación no es un algoritmo nuevo, sino la integración de todas estas piezas en un robot que funciona.

> **Idea clave:** *Cuatro ideas: objetivos, arbitraje, bajo cómputo, lección. Cierra con «la aportación es la integración».*

PARA ENTENDERLO

- **Aportación = integración:** no hay un algoritmo nuevo; el valor está en que seguimiento, gestos, mapa y navegación funcionen juntos en un robot de bajo coste.
- **Bajo cómputo:** ~30 % por núcleo en el NUC con todo en marcha.
- **Lección:** casi todos los errores importantes solo aparecieron en el robot real y se resolvieron midiendo.
- Cierra mirando al tribunal, sin leer.

4 de 5 | **~30 % por núcleo**

09 Seguimiento y seguridad

≈ 1:15 · ACABA EN 9:00

QUÉ DECIR

Las detecciones son ruidosas e intermitentes, así que tracking_node mantiene un filtro de Kalman de seis estados con aceleración constante. Una puerta de Mahalanobis rechaza los saltos inverosímiles y un controlador PD orienta el robot hacia la persona.

La gráfica muestra por qué importaba. En la primera prueba en movimiento, la persona solo se detectaba el **32 %** del tiempo. Tres correcciones encadenadas —un filtro de continuidad, la puerta de Mahalanobis y un límite a la velocidad angular— lo subieron al **83 %** y redujeron los saltos de posición del 3,5 al **0,7 %**.

Para la seguridad, el robot vigila un sector frontal de ±**50 grados**: frena, se detiene y, si sigue bloqueado, hace un breve rodeo. En las primeras tomas con obstáculos rozó el mobiliario dos veces, porque el factor de reducción nunca llegaba a cero; tras corregirlo, no hubo más contactos con muebles.

Idea clave: *Kalman de 6 estados + Mahalanobis + PD. Detección en movimiento 32 % → 83 %. Seguridad: sector de ±50°, frena/para/rodea; 2 roces → 0 tras corregir el factor.*

PARA ENTENDERLO

- **Paso a paso en la ficha siguiente** («por dentro»).
- **Kalman de 6 estados** [x, y, vx, vy, ax, ay]: predice dónde estará la persona y suaviza la medida ruidosa.
- **Puerta de Mahalanobis:** descarta medidas incompatibles con la predicción (umbral χ^2 al 95 %).
- **PD de rumbo:** gira el robot hacia la persona; la parte D amortigua las oscilaciones.
- **8/07, primera prueba en movimiento:** detección 32,3 % → 71,7 % (filtro de continuidad) → 82,6 % (Mahalanobis + límite de velocidad angular). Saltos > 0,8 m: 3,5 % → 0,7 %.
- **Seguridad:** sector de ±50°, frenado entre 0,40 m y 0,25 m, rodeo si sigue bloqueado. 2 roces el 21/07 → corregido el 22/07 → sin contactos con muebles.

32 % → 83 % | **saltos 3,5 % → 0,7 %** | **±50°**

19 Muchas gracias

≈ 0:10 · ACABA EN 17:50

QUÉ DECIR

Muchas gracias por su atención. Quedo a su disposición para las preguntas.

Idea clave: *Sonríe y espera; no rellenes el silencio.*

PARA ENTENDERLO

- Pausa, reformula la pregunta y responde en 2–3 frases. Más detalle solo si lo piden.
- **Reserva** (escribe el número + Intro): 21 máquina de estados · 22 Nav2 · 23 resultados por escenario · 24 obstáculos · 25 sistemas TF · 26 planificación y coste · 27 IA.
- Si no sabes algo: di qué mediste, cuál crees que es la causa y cómo lo comprobarías.
- Preguntas probables con su respuesta: fichas del final.

POR DENTRO

09 Seguimiento y seguridad, por dentro

PARA ESTUDIAR Y PARA LAS PREGUNTAS · 1 DE 2

Lo que hace tracking_node en cada ciclo (≈ 10 veces por segundo, una por escaneo del LiDAR):

1. **Llega la medida:** detection_node publica la posición (x, y) de la persona en el sistema del robot. A veces falta o trae ruido.
2. **Predicción (Kalman):** estado [x, y, vx, vy, ax, ay] con modelo de aceleración constante: predice dónde estará la persona y con qué incertidumbre. Q = 0,01 (cuánto puede cambiar su movimiento) y R = 0,15 (cuánto se fía de la medida): R alto → más suavizado.
3. **Puerta de Mahalanobis:** mide lo lejos que está la medida de la predicción en unidades de incertidumbre. Si supera el umbral χ^2 al 95 % (2 g. l., ≈ 5,99) se descarta como salto. Si varias seguidas quedan fuera, el filtro se reinicia en la nueva posición: la persona ha reaparecido de verdad.
4. **Corrección:** mezcla predicción y medida según sus incertidumbres (ganancia de Kalman, forma de Joseph). Sin medida, usa solo la predicción → no frena en seco. Más de 2 s sin persona → se detiene.
5. **Velocidad lineal:** según la distancia, con rampa suave (exponente 1,5) hacia la distancia objetivo de 1,0 m; máximo 0,18 m/s y aceleración limitada por ciclo.

Preguntas probables

1 DE 4 · RESPUESTA COMPLETA · [APARTADO DE LA MEMORIA]

P01 ¿Por qué el gesto del tejado y no otro?

Porque es simétrico: se reconoce igual de frente que de espaldas, y mientras el robot sigue, la persona está de espaldas a él. No se parece a nada que se haga al caminar y recuerda a una casa. Descarté los brazos cruzados porque las muñecas se tapan entre sí, y las dos manos arriba sin juntarlas se confundían con los gestos de una mano. [§2.6, §3.3.2]

P02 ¿Por qué MediaPipe Pose y no un detector más pesado?

El NUC no tiene GPU. MediaPipe Pose, basado en BlazePose, da 33 puntos clave en tiempo real en CPU, y solo necesito hombros, muñecas y cabeza. El detector HOG original necesitaba el cuerpo entero en el encuadre, algo imposible a un metro. [§2.4]

P03 ¿Por qué reimplementaste DBSCAN?

La extensión compilada de scikit-learn no era compatible con la versión de Python del NUC. Mi versión sobre el KD-tree de scipy mantiene la misma semántica y, con unos cientos de puntos por escaneo, el coste es despreciable. eps = 0,12 m y mínimo 4 puntos se calibraron con la resolución angular real del LiDAR. [§3.3.1, §4.3]

P04 ¿Por qué no un detector de piernas con aprendizaje profundo?

No hay GPU a bordo, y el detector clásico ya estaba validado en la Parte 1, así que pude reutilizarlo y mejorarlo. Las redes convolucionales sobre escaneos 2D existen y serían una posible extensión. [§2.3]

P05 ¿Por qué un filtro de Kalman lineal y no un EKF, un UKF o un filtro de partículas?

Tanto el modelo de movimiento, de aceleración constante, como la observación, una posición en coordenadas cartesianas, son lineales, así que basta con el Kalman lineal, que es la opción más barata. Un EKF o un UKF solo compensaría con una observación no lineal, como distancia y ángulo. Los valores atípicos los resuelve la puerta de Mahalanobis, con el umbral chi-cuadrado del 95 %. [§2.7, §4.5]

POR DENTRO

09 Seguimiento y seguridad, por dentro

PARA ESTUDIAR Y PARA LAS PREGUNTAS · 2 DE 2

6. **Giro (PD):** error de ángulo → P (Kp = 2,0) + D (Kd = 0,3, amortigua). Zona muerta de ±8°, menos giro a más velocidad, atenuación de cerca, arranque suave de 1,5 s y cambio máximo por ciclo (0,3 rad/s).
7. **Evitación reactiva:** sector frontal de ±50°: cada punto a menos de 0,40 m «empuja» el giro hacia el lado libre, con más peso cuanto más cerca y más de frente; el giro final es la media ponderada.
8. **Factor de reducción:** multiplica la velocidad lineal: 1 a 0,40 m → 0 a 0,25 m (parada). Medido desde el LiDAR; el borde del robot está 0,15 m por delante → para a ~10 cm del contacto.
9. **Maniobra de rodeo:** si el factor sigue < 0,5 durante 1,5 s: giro cerrado de 1,5 s (0,5 rad/s) hacia el lado de la evasión + avance recto de 2,5 s a 0,12 m/s. Si el factor vuelve a caer, se cancela.

La historia de las cifras. 8/07: primera prueba en movimiento, 32 % de detección, saltos y giro casi siempre saturado → filtro de continuidad (72 %) → puerta de Mahalanobis + límite angular (83 %). La saturación la redujo después la atenuación por proximidad (15/07). 21/07: dos roces porque el factor nunca bajaba de 0,4 (un límite que no se aplicaba) → 22/07: fórmula nueva, umbral de 0,35 a 0,40 m y rodeo → sin contactos con muebles.

Preguntas probables

2 DE 4 · RESPUESTA COMPLETA · [APARTADO DE LA MEMORIA]

P06 ¿Por qué una fusión asimétrica en lugar de una fusión LiDAR–cámara completa?

Evita una calibración extrínseca precisa: solo necesito el campo de visión horizontal de la cámara y un signo. El LiDAR sigue siendo la fuente principal de posición; la cámara solo elige un clúster cuando no hay par de piernas. La contrapartida es que, en los giros, los dos sensores comparten el mismo punto ciego frontal. [§2.5]

P07 ¿Cómo evitas que el seguidor y Nav2 se peleen por el robot?

Nav2 nunca publica directamente a la base: su salida se reasigna a /nav2/cmd_vel. control_node recibe las dos y solo reenvía la del estado activo. Y tiene que ser por estado: el seguidor inactivo sigue publicando velocidad cero a 10 Hz, lo que habría anulado a Nav2. [§3.3.4, §3.8, §4.6]

P08 ¿Por qué fallaron 2 de las 11 vueltas a casa?

No fue por el gesto ni por la máquina de estados. La estimación de AMCL, con una incertidumbre de medio metro, situó al robot dentro de la zona de seguridad de un obstáculo del mapa, así que el planificador no encontró una ruta válida y el robot volvió a IDLE de forma segura. La solución natural es una maniobra de salida antes de pedir la ruta, junto con un mejor ajuste de AMCL. [§5.8, §5.9]

P09 ¿Qué pasa si hay un obstáculo en el camino a casa?

Nav2 lo ve con el LiDAR en su costmap local y lo rodea. Lo probé el 27 de julio con un obstáculo que no estaba en el mapa, y se detectó y esquivó; también aparece en el vídeo 3. [§5.7]

P10 ¿Qué significa exactamente «cancelación en 20 ms»?

Cuando levanto la mano izquierda durante HOMING, control_node cancela la meta de Nav2 y Nav2 confirma la cancelación en unos 20 milisegundos; el robot avanzó unos 7 centímetros más antes de parar. [§5.8]

P11 La detección mejoró del 32 % al 83 %, pero ¿mejoró la saturación angular?

No de forma consistente, y la memoria lo dice: esas tres correcciones actúan sobre la continuidad de la detección, no sobre la ganancia del controlador angular. Lo que redujo la saturación fue la atenuación por proximidad, añadida después, el 15 de julio. [§5.4]

VÍDEO 2 · 30–40 S

10 Obstáculos sólidos

≈ **0:40** · ACABA EN **9:40**

QUÉ DECIR

En este vídeo el robot me sigue sorteando mobiliario sólido.

▶ **Reproducir el vídeo.** Se ve cómo frena cuando el armario entra en su sector frontal, y cómo lo rodea.

Idea clave: *Señala el frenado y el rodeo cuando ocurran.*

PARA ENTENDERLO

- Vídeo recortado a ~30–40 s.
- Cuando el mueble entra en el sector frontal (±50°, a menos de 0,40 m) el robot reduce la velocidad; si sigue bloqueado más de 1,5 s, hace el giro cerrado y avanza para rodearlo.
- Si la persona se acerca mucho, la evitación también reacciona: no distingue persona de obstáculo (limitación conocida, diapositiva 17).

Preguntas probables

3 DE 4 · RESPUESTA COMPLETA · [APARTADO DE LA MEMORIA]

P12 ¿Qué pasa si el robot pierde a la persona?

En pérdidas cortas, el filtro de Kalman sigue prediciendo la posición, así que el robot no frena de golpe. Si la persona desaparece durante más de unos dos segundos, el seguidor detiene el robot, y control_node vuelve a IDLE tras 1,5 segundos de pérdida. [§3.3.3, §4.6]

P13 ¿Por qué no se detectó la silla de patas finas?

El LiDAR mide un único plano a unos 47 centímetros: apenas ve las patas finas de una silla y nunca el asiento. Es una limitación del sensor confirmada con los datos; la solución es la cámara RGB-D Orbbec Astra disponible o un segundo LiDAR a otra altura. [§5.6]

P14 ¿Son tus resultados estadísticamente significativos?

No, y lo digo en la memoria: varios resultados vienen de una sola toma, y Nav2 y el gesto «casa» se validaron en una sesión cada uno. Son primeros resultados en el robot real; repetirlos con más muestras, personas y entornos es la primera línea de trabajo futuro. [§5.9, §5.10]

P15 ¿Cómo sabes que una corrección no rompió lo que ya funcionaba?

Todas las tomas se analizaron con el mismo procedimiento —rosbag, CSV, métricas y figuras—, así que cada cambio se comparó con números. Las últimas correcciones también se comprobaron fuera de línea con el código real de los nodos y escenarios sintéticos antes de volver al robot. [§5.1]

P16 ¿Por qué no usaste simulación?

Casi todos los errores relevantes —el signo invertido del PD, el sector de evitación invertido, el factor de reducción, la latencia de DDS— solo aparecieron en el robot real. Con sesiones de laboratorio limitadas, prioricé las pruebas reales; la simulación seguiría siendo útil para pruebas de regresión. [§6.3]

P17 ¿Cuál es exactamente tu aportación respecto a la Parte 1?

La Parte 1 detectaba piernas con el LiDAR y las seguía. Yo añadí los gestos, el respaldo con la cámara, el filtro de Kalman de seis estados con la puerta de Mahalanobis y el control PD, la evitación de obstáculos corregida con el rodeo, el mapa SLAM, AMCL y Nav2 y, sobre todo, el arbitraje de control_node con el estado HOMING; todo validado en el robot real. [§1.1, §2.10, §6.2]

11 6 / 7 metas alcanzadas

≈ 1:10 · ACABA EN 10:50

QUÉ DECIR

Para la parte espacial usé las herramientas estándar de ROS 2. SLAM Toolbox construyó este mapa del laboratorio; AMCL localiza el robot sobre él; y Nav2 planifica la ruta, que sigue el controlador DWB.

En la primera prueba, AMCL parecía congelado. La causa era la latencia de descubrimiento de DDS: mis comandos de prueba terminaban antes de que los nodos se descubrieran entre sí. Con comandos más largos, convergió.

Después se alcanzaron **seis de siete** metas —un **86 %**—, incluidas dos rutas de unos ocho metros y una con un obstáculo que no estaba en el mapa, que el costmap local detectó y esquivó. El único fallo llegó al cambiar la meta a mitad de ruta, y Nav2 se recuperó en unos dos segundos.

> **Idea clave:** *Mapa con SLAM Toolbox, localización con AMCL, Nav2 con DWB. 6/7 metas (86 %), rutas de 8 m, obstáculo no mapeado esquivado. Anécdota de DDS.*

PARA ENTENDERLO

- **Paso a paso en la ficha siguiente** («por dentro»).
- **Tres piezas:** SLAM Toolbox hace el mapa (una vez) · AMCL dice dónde está el robot en ese mapa (siempre) · Nav2 planifica y sigue la ruta (para cada meta).
- **27/07, 7 metas enviadas desde RViz:** ≈ 3,7 · 5,9 · 5,3 (con obstáculo real no mapeado) · 8,0 · 8,1 · 0,6 (cambio de meta: fallo) · 3,6 m.
- **Único fallo:** al cambiar de meta a mitad de trayecto; lo recuperó el gestor de ciclo de vida de Nav2 en ~2 s y no se repitió.
- Ese día se volvió a grabar el mapa porque el guardado no reflejaba bien el laboratorio; todos los resultados son con el mapa nuevo.

6 de 7 (86 %)	~8 m	~2 s

Preguntas probables

4 DE 4 · RESPUESTA COMPLETA · [APARTADO DE LA MEMORIA]

P18 ¿Qué pasa si hay varias personas?

El sistema está pensado para seguir a un único usuario. La puerta de Mahalanobis y los filtros de continuidad mantienen la pista sobre la misma persona, pero no hay reidentificación: alguien que cruce muy cerca podría capturar la pista. El seguimiento de varias personas queda fuera del alcance de este TFM. [§2.3, §2.7]

P19 ¿Qué carga soporta el NUC?

Con todo en marcha, en torno al 30 % por núcleo en sus cuatro núcleos. En las primeras fases hubo desconexiones y bloqueos de puertos, que se resolvieron eliminando puertos duplicados, bajando las frecuencias de publicación y simplificando la detección. [§5.9]

P20 ¿Cómo usaste las herramientas de IA?

Como apoyo técnico: diagnóstico de errores y registros de ROS 2, revisión de código, reformulación de la memoria y trazado de las figuras a partir del código y de los datos grabados. La arquitectura, las decisiones de diseño, los experimentos y la interpretación de los resultados son míos. Está declarado en el Apéndice A. [Apéndice A]

P21 ¿Qué harías primero con más tiempo?

Una maniobra de salida para que el robot se aleje de los obstáculos antes de pedir la ruta a Nav2 —eso resolvería los dos fallos de vuelta a casa— y, después, el guiado a varios destinos, con un gesto o un código QR por destino. [§5.10]

P22 ¿Cuánto costó?

Unos 5.400 € en total, casi todo tiempo de personal: unas 350 horas. El hardware ya estaba en el laboratorio; solo se cuenta su amortización durante cinco meses. [§1.6]